

COMS BC1016

Introduction to Computational Thinking and Data Science

Lecture 2: Introduction to Python

BARNARD COLLEGE OF COLUMBIA UNIVERSITY

Sept 30, 2025

Announcements

Lab Updates

- Way too many people have conflicts on Thursdays :(
- We cancelled one of the Thursday labs and opened a second Wed 4:10 section
- Section 1: Wednesday 2:40-3:55 (516 Milstein)
- Section 2: Wednesday 4:10-5:25 (516 Milstein)
- Section 3: Thursday 9:40-11 (516 Milstein)
- **Section 4: Wednesday 4:10-5:25 (LL107 R&D Science Center)**
- Labs start this week! Make sure you are registered for a lab!

Lab Updates

If you are not currently registered for 1017 and **Wed 2:40** is the only section you are able to make, email me ASAP

- Section 1: Wednesday 2:40-3:55 (516 Milstein)
- Section 2: Wednesday 4:10-5:25 (516 Milstein)
- Section 3: Thursday 9:40-11 (516 Milstein)
- Section 4: **Wednesday 4:10-5:25** (LL107 R&D Science Center)
- Labs start this week! Make sure you are registered for a lab!

We need to know if Section 1 also needs to be expanded

Schedule, slides, and useful links on the course website: <https://www.eysalee.com/courses/f26/bc1016.html>

Lecture Schedule

The schedule below will be updated as the course progresses.

Week	Date	Topic	Lab	Homeworks	Due Dates
1	9/9	1 - Introduction [Slides - Updated 9/10]	<i>No Lab</i>		
2	9/14	2 - Intro to Python			
	9/16	3 - Tables	Lab 1 - Expressions		
3	9/21	4 - Data Visualizations		HW 1 - Arrays and Tables	
	9/23	5 - Histograms	Lab 2 - Table Operations		
4	9/28	6 - Functions		HW 2 - Table Manipulation and Visualization	
	9/30	7 - Groups	Lab 3 - Data Types and Arrays		HW 1 Due

How to Approach Programming in BC1016

Programming is all about practice

- Class: Demos introduce terms & rules
- Labs & Homework: Try out programming yourself

How to Approach Programming in BC1016

Programming is all about practice

- Class: Demos introduce terms & rules
- Labs & Homework: Try out programming yourself

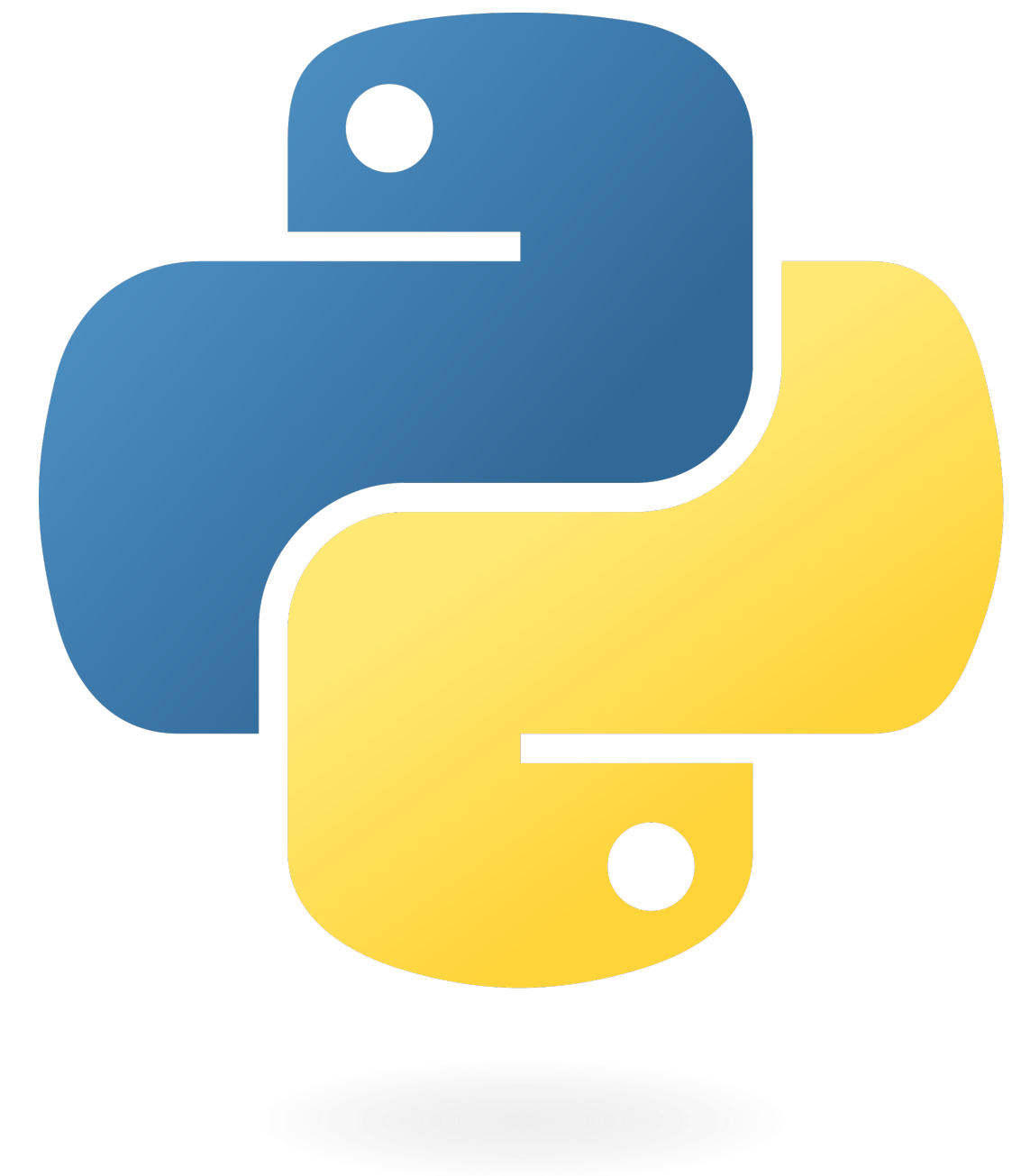
As you learn to program, the goal is **not** to memorize every command!

Instead, we will focus on **reading** code, **predicting** what it will do, **testing**, and then **fixing** it if it behaves differently from what we intended

Basics of Python

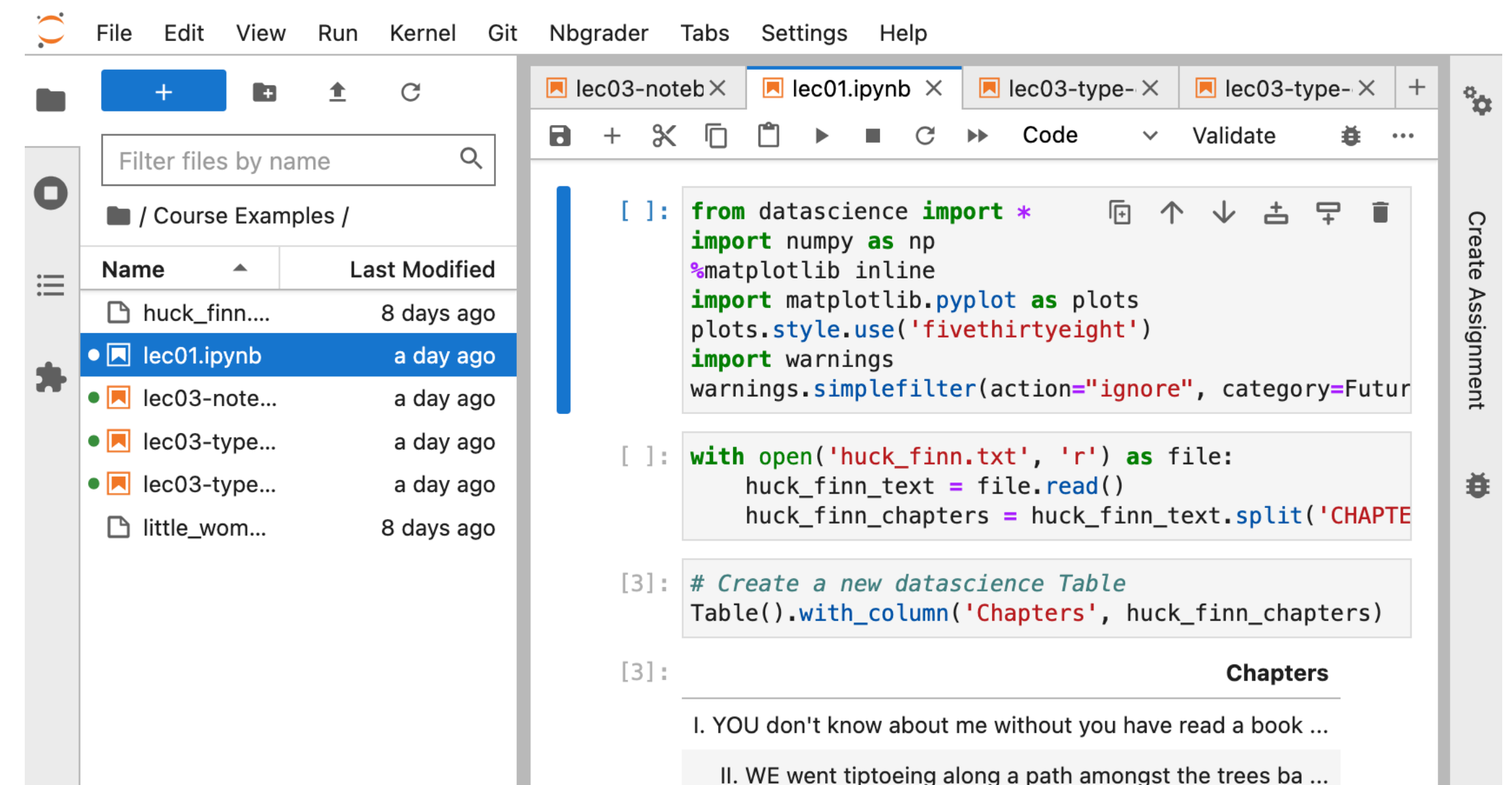
Python Programming Language

- Popular programming for software development
- Especially popular for data science
- Learning programming is about learning how to think computationally
 - This skills transfers to other languages



Jupyter Notebooks

- Can be run locally or in cloud-based environments
- For this class: JupyterHub Server
- Benefits of cloud-based
 - Access anywhere
 - Expandable compute resources



Let's poke around in Jupyter...

Demo: Let's make a Jupyter Notebook

(Python) Terminology

- **Syntax** refers to the rules and grammar of a programming language
 - If you ever see a `SyntaxError`, something violated these rules
- In Python, an **expression** is code that evaluates to a value
- In Python, a **statement** executes code (performs actions) but does not result in a value

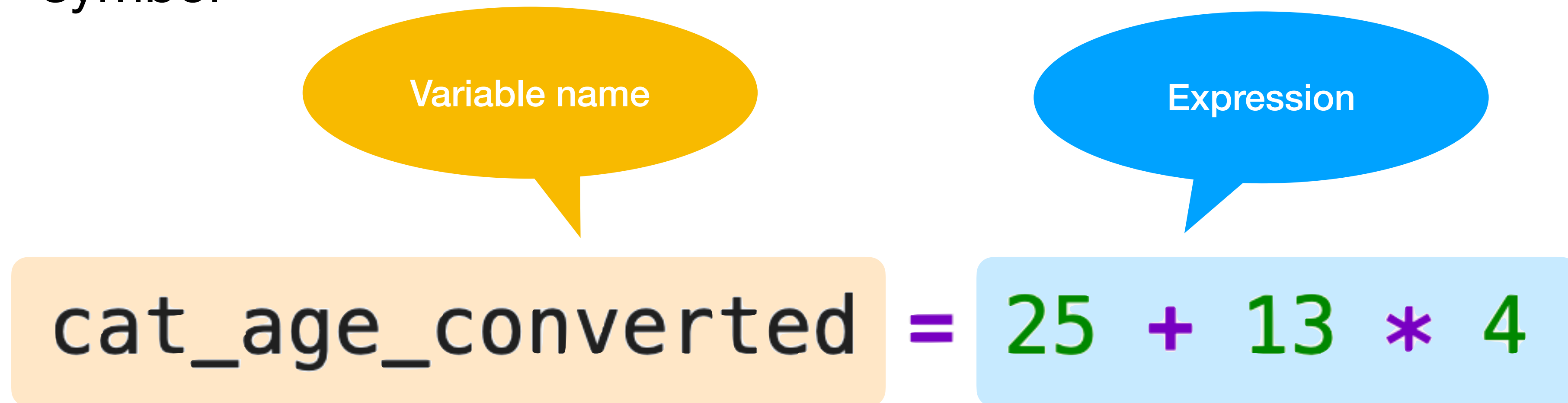
Variables and Assignments

- **Variables** are values you can assign values to
 - “Variable” because they can change
- An **assignment** statement (=) changes the value of the name to the left of the = symbol

```
cat_age_converted = 25 + 13 * 4
```

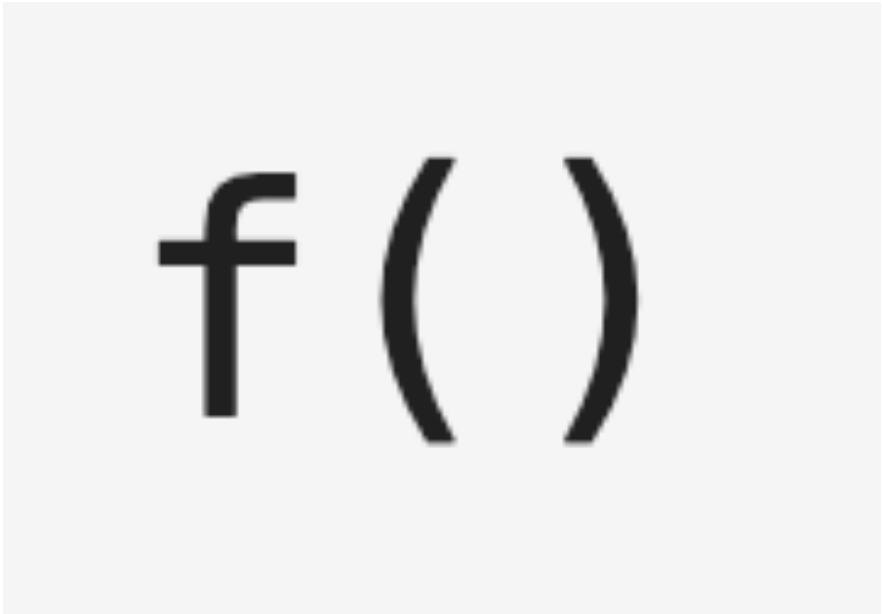
Variables and Assignments

- **Variables** are values you can assign values to
 - “Variable” because they can change
- An **assignment** statement (=) changes the value of the name to the left of the = symbol



Functions

- **Functions** are reusable blocks of code
 - To run a function, you type the function name following by parenthesis



```
f()
```

Functions

- **Functions** are reusable blocks of code
 - To run a function, you type the function name following by parenthesis
 - Sometimes functions can take expressions as inputs (“**arguments**”)

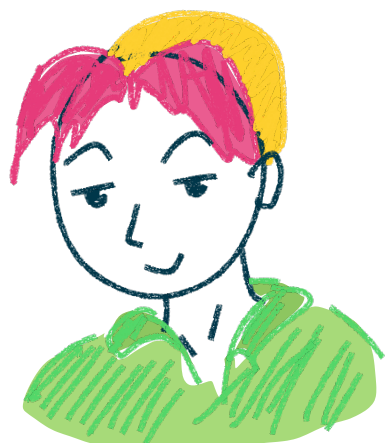


```
abs(-1)
```

1

Functions

- **Functions** are reusable blocks of code
 - To run a function, you type the function name following by parenthesis
 - Sometimes functions can take expressions as inputs (“**arguments**”)
 - Multiple arguments are separated by a comma



```
abs(-1)
```

1

```
max(4, 200, 7)
```

200

Functions

- **Functions** are reusable blocks of code
 - To run a function, you type the function name following by parenthesis
 - Sometimes functions can take expressions as inputs (“**arguments**”)
 - Some useful functions aren’t built in, so we tell Python where to find them using **import**

```
import numpy  
numpy.sqrt(4)
```

```
2.0
```

Demo: Functions

Data Types

Numbers

- **Integers**: Whole numbers (e.g., 3, -10, 25)
- **Floats**: Anything with decimals (e.g., 3.1, -10.2, 2.0)
 - For very large numbers, may show up in scientific notation
 - Limitations of floats:
 - Limited size (but the limit is huge)
 - Limited precision of 15-16 decimal places
 - After arithmetic, final few decimal places can be wrong

Demo!

Strings

- **Strings** are snippets of text in python
- You can specify a string using either single quotes or double quotes
 - `"a"`
 - `'This is a sentence'`
 - `"This is another sentence. Wow!"`

Demo!

Conversions

- Strings that contain numbers can be converted to numbers:
 - `int('12')`
 - `float('1.2')`
- Any value can be converted to a string
 - `str(5)`
- Numbers can be converted to other numeric types
 - `float(1)`
 - `int(1.2)`

Discussion Question

Suppose we run the following statements:

```
x = 3
```

```
y = '4'
```

```
z = '5.6'
```

What is the source of the error in each example?

A. `x + y`

B. `x + int(y + z)`

C. `str(x) + int(y)`

D. `y + float(z)`

Booleans

- True or False values
- Useful for conditional statements
 - Usually of the form “if some statement is true, then execute some code. Otherwise, do something else.”
- We will talk more about Booleans in a few weeks

Arrays

- **Arrays** are sequences of values

(Note: in this class we'll mostly use NumPy arrays, which we will cover in more detail next lecture)

- **Examples:**
 - `[1, 2, 3, 5]`
 - `["Mystery", "Abby", "Jinu", "Baby", "Romance"]`
 - `['apples', 'bananas', 'apples', 'apples']`
- Elements of an array should have the same type

Arrays

Demo!

- Arrays are **zero-indexed**
 - The first element is the 0th and the last is length-1
- Arithmetic is applied to each element individually
- Adding two arrays adds the corresponding elements (but they must be the same length!)

```
from datascience import *  
onetwothree = make_array(1,2,3)  
onetwothree * 2
```

```
array([2, 4, 6])
```

```
from datascience import *  
onetwothree = make_array(1,2,3)  
twothreefour = make_array(2,3,4)  
onetwothree + twothreefour
```

```
array([3, 5, 7])
```

Every Value has a Type

Type	Example
Int	3
Float	3.0
String	'three'
Boolean	True
Arrays	[1, 2, 3, 4]
Functions	abs(-5)

Every Value has a Type

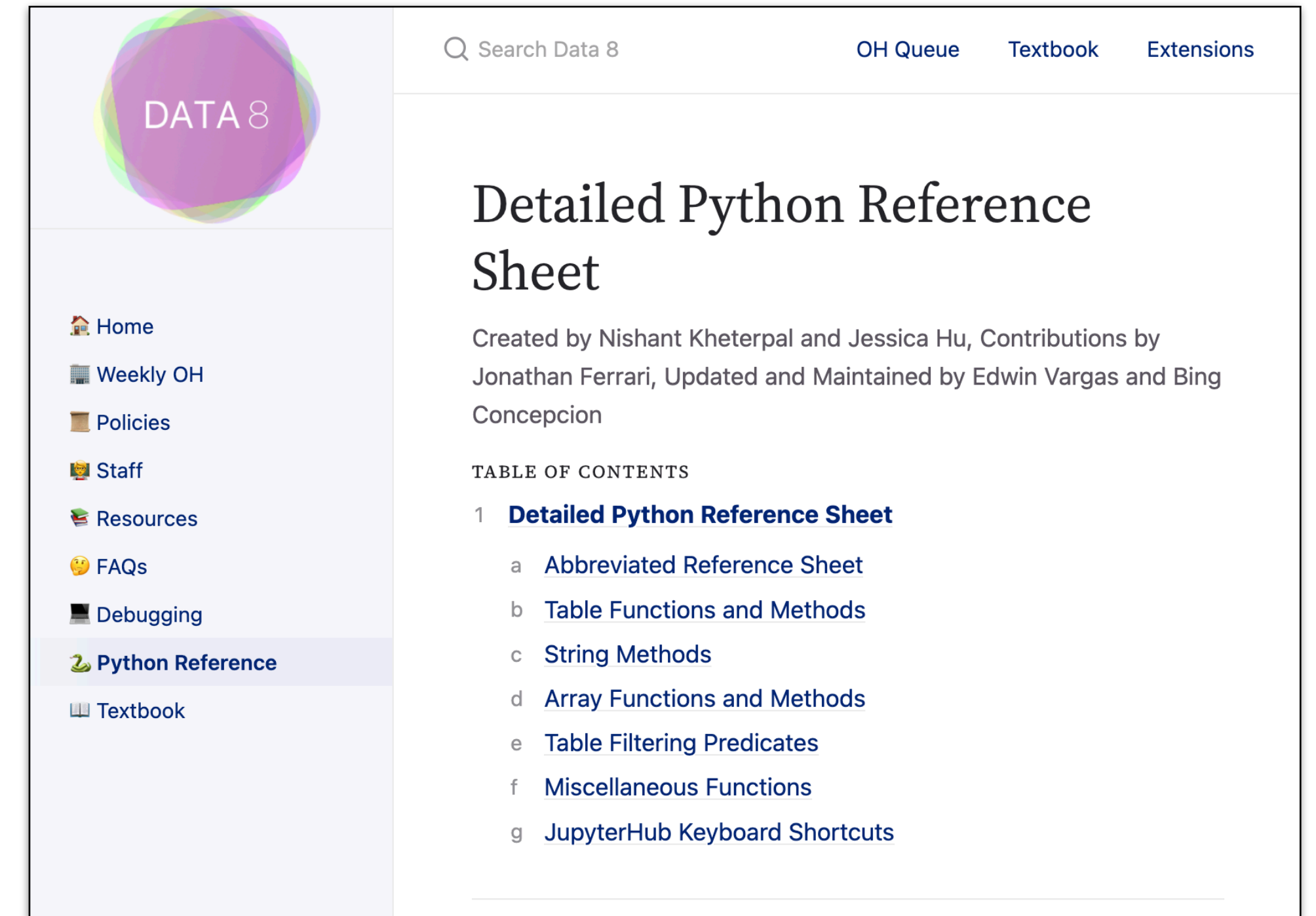
- The `type` function can tell you the type of a value
 - `type(2)`
 - `type(2 + 2)`
- An expression's type is based on its value, not how it looks
 - Example: What does this second line evaluate to?

```
two = 2
```

```
type(two)
```

Python Reference

<https://www.data8.org/sp25/reference/>



The screenshot shows the Data 8 website's Python Reference page. The header includes a search bar, "OH Queue", "Textbook", and "Extensions" links. A sidebar on the left lists navigation options: Home, Weekly OH, Policies, Staff, Resources, FAQs, Debugging, Python Reference (highlighted), and Textbook. The main content area is titled "Detailed Python Reference Sheet" and credits Nishant Kheterpal, Jessica Hu, Jonathan Ferrari, Edwin Vargas, and Bing Concepcion. It features a "TABLE OF CONTENTS" with links to various reference sheets and methods.

DATA 8

Search Data 8 OH Queue Textbook Extensions

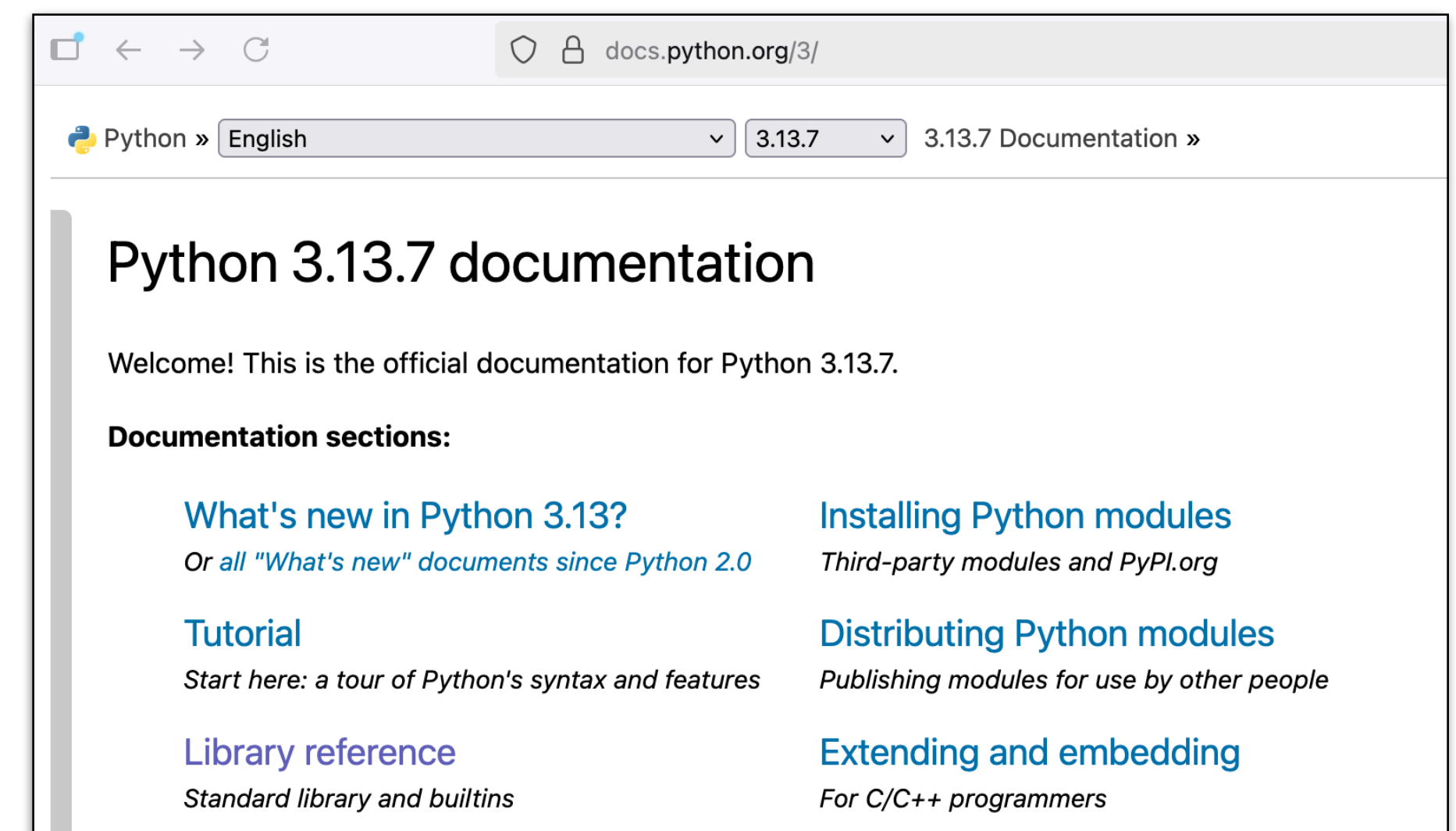
Detailed Python Reference Sheet

Created by Nishant Kheterpal and Jessica Hu, Contributions by Jonathan Ferrari, Updated and Maintained by Edwin Vargas and Bing Concepcion

TABLE OF CONTENTS

- 1 **Detailed Python Reference Sheet**
 - a Abbreviated Reference Sheet
 - b Table Functions and Methods
 - c String Methods
 - d Array Functions and Methods
 - e Table Filtering Predicates
 - f Miscellaneous Functions
 - g JupyterHub Keyboard Shortcuts

<https://docs.python.org/3/>



The screenshot shows the official Python 3.13.7 documentation page. The browser address bar shows "docs.python.org/3/". The page has a header with "Python »", a language dropdown set to "English", a version dropdown set to "3.13.7", and a link to "3.13.7 Documentation ». The main heading is "Python 3.13.7 documentation". Below it, a welcome message states: "Welcome! This is the official documentation for Python 3.13.7." A section titled "Documentation sections:" lists several links with descriptions.

Python » English 3.13.7 3.13.7 Documentation »

Python 3.13.7 documentation

Welcome! This is the official documentation for Python 3.13.7.

Documentation sections:

- [What's new in Python 3.13?](#)
Or all "What's new" documents since Python 2.0
- [Installing Python modules](#)
Third-party modules and PyPI.org
- [Tutorial](#)
Start here: a tour of Python's syntax and features
- [Distributing Python modules](#)
Publishing modules for use by other people
- [Library reference](#)
Standard library and builtins
- [Extending and embedding](#)
For C/C++ programmers

Next Class

- Today
 - Jupyter Notebooks
 - Data Types
- Wednesday
 - Tables
- Monday
 - Charts & Visualization